The GRADIEND Python Package:
An End-to-End System for Gradient-Based Feature Learning

EMNLP 2026 System Demonstration · Jonathan Drechsel, Steffen Herbold · University of Passau

Learn, evaluate, rewrite and compare feature directions in language models

The Python package ***gradiend*** implements **GRADI**ent **EN**coder **D**ecoder (GRADIEND) [1]

Tested on many feature types: grammatical, semantic, social, factual, language, algorithmic and more

Supports: encoder-only and decoder-only models, multi-class and present-vs-absent features

Also included: built-in plotting functionality, multi-seed mode, pruning (about 3x less training memory)

Python package for Transformer models

Python · PyTorch · Hugging Face

Targeted feature learning

Unlike Sparse Autoencoders (SAEs), you decide which feature is learned: two classes of inputs define it, e.g., male/female for *gender* or positive/negative for *sentiment*.

Hugging Face Trainer-style API

Similar interface to the Hugging Face Trainer which also works with Hugging Face models and datasets: `TrainingArguments`, `train()`, `evaluate()`.

Feature comparison at scale

How similar are different features encoded in a language model? Top-k weight overlap and cross-encoding compare many learned features without retraining.

Workflow and Demo 1: English pronouns (3SG vs. 3PL) with bert-base-cased

1 Feature selection and data generation

Two classes define the feature axis, e.g. 3rd-person singular vs. plural (3SG vs. 3PL).

Text The nurse Mary said that `[MASK]` was exhausted.
Factual `[MASK]` = “she”
Counterfactual `[MASK]` = “they”

`TextPredictionDataCreator`

```
from gradiend import *

SG = ["he", "she", "it"]
PL = ["they"]
creator = TextPredictionDataCreator(
    base_data="wikimedia/wikipedia",
    hf_config="20231101.en",
    feature_targets=[
        TextFilterConfig(SG, id="3SG"),
        TextFilterConfig(PL, id="3PL")],
    min_left_context_words=10)
data = creator.generate_training_data(
    max_size_per_class=2500)
excl = ["i", "we", "you"]
neutral = creator.generate_neutral_data(
    additional_excluded_words=excl,
    max_size=1000)
```

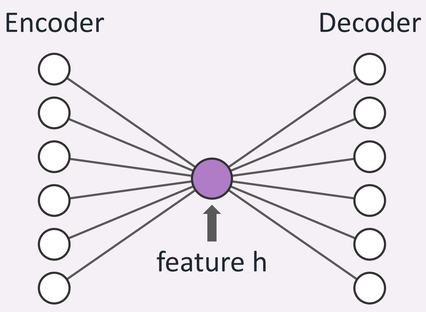
Feature-inducing data: the package masks texts (as above) and their feature-inducing MLM or CLM gradients control which feature is learned.

Targets: a masked word is the factual target and a word of the other class its counterfactual target.

Neutral data: unrelated words that must stay apart from the feature classes.

Target matching: word lists with optional spaCy POS tags to tell apart ambiguous forms, also multi-word targets.

2 GRADIEND training

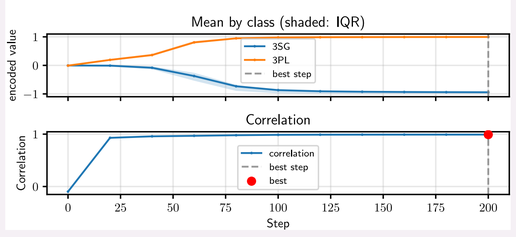


Counterfactual gradient encoded as scalar feature value

Gradient difference decoded as model update direction

`Trainer.train()`

```
pre = PrePruneConfig(n_samples=8, topk=0.1)
post = PostPruneConfig(topk=0.01)
args = TrainingArguments(
    train_batch_size=8, max_steps=200,
    eval_steps=20, learning_rate=1e-5,
    pre_prune_config=pre,
    post_prune_config=post)
trainer = TextPredictionTrainer(
    model="bert-base-cased", data=data,
    eval_neutral_data=neutral, args=args)
trainer.train()
trainer.plot_training_convergence(
    class_spread="iqr")
```

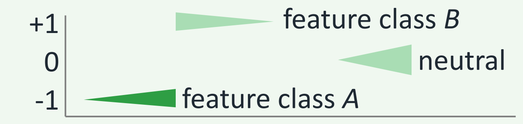


Training: the encoder separates the classes (means move to -1 / +1).

Pruning: optional pre- and post-pruning of dimensions reduces memory and model size.

3 Intra-model evaluation

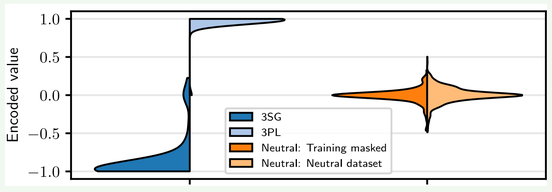
Encoder analysis: are feature classes separated?



Decoder analysis: shift of feature-token probabilities, constrained by the language modeling score (LMS).

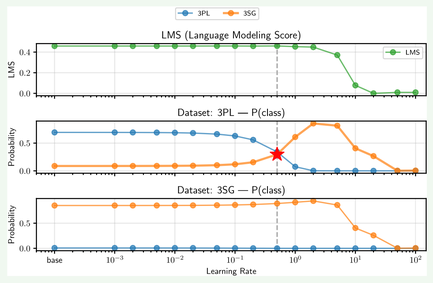
`Trainer.evaluate()`

```
enc = trainer.evaluate_encoder(plot=True)
print(enc["correlation"])  # 0.9688
```



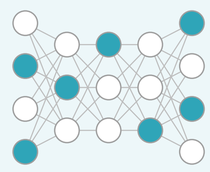
Class means: +0.97 (3PL), -0.86 (3SG)

```
dec = trainer.evaluate_decoder(
    plot=True, target_class="3SG")
lr = dec["3SG"]["learning_rate"]  # 0.5
```



4 Model rewrite

Modify model behavior along the learned feature direction, i.e. strengthen or weaken the feature's behavior.



`Trainer.rewrite_base_model()`

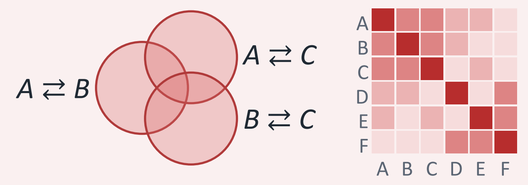
```
changed = trainer.rewrite_base_model(
    decoder_results=dec,
    target_class="3SG")
```

Why rewrite? Unlike activation steering, the edit is in the weights: no hooks at inference, so it is easy to save and share.

Strength: set automatically from step 3 (LMS-constrained) or manually.

5 Inter-model evaluation

Compare multiple GRADIEND models.



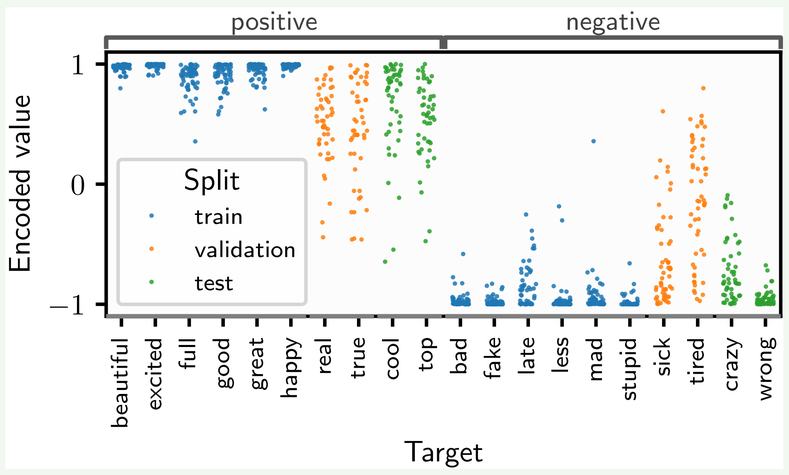
`plot_topk_overlap()`

```
models = {t.run_id: t.get_model()
          for t in trainers}
plot_topk_overlap(models, topk=1000)
```

Top-k weight overlap: how many top weights two GRADIEND models of the same language model share. No extra training, so it scales to many features (Demo 3).

Demo 2: Semantic features

Generalization to unseen words

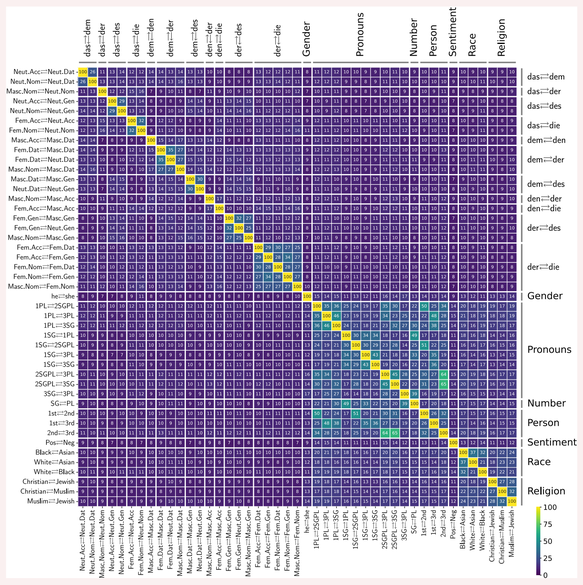


Semantic feature: positive vs. negative sentiment, 10 adjectives per class.

Generalizes: words never seen during training (validation, test) are mostly still encoded on the correct side, although less strongly.

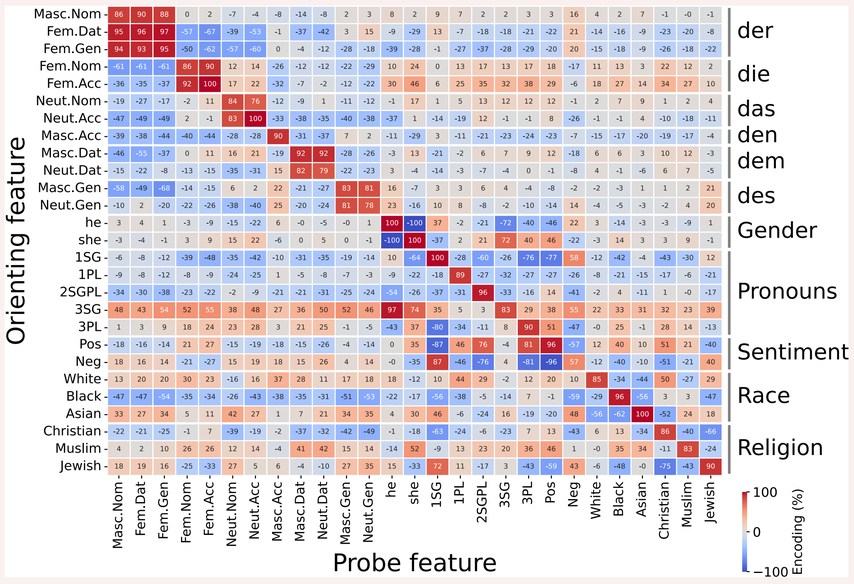
Demo 3: Large-scale feature comparison

Top-k weight overlap



Which features share weights? 45 pairwise runs (multilingual BERT). Related features such as German articles overlap most.

Cross-encoding



How does the model of one feature encode another feature? Rows: the feature a model was trained for. Columns: the feature whose inputs are encoded. Shown for Qwen-2.5 0.5B.

PyPI
Install in one line
`pip install gradiend`

GitHub
Apache 2.0 license
`aieng-lab/gradiend`

Docs
Guides and examples
`aieng-lab.github.io/gradiend`

Paper
This system paper
`arxiv.org/abs/2602.23993`

[1] Method
GRADIEND, ICLR 2026
`openreview.net/forum?id=1vBNAnAgCD`

Paper: https://arxiv.org/abs/2602.23993
Code: https://github.com/aieng-lab/gradiend

Contact: Jonathan Drechsel, jonathan.drechsel@uni-passau.de
https://www.fim.uni-passau.de/ai-engineering

EMNLP2026 BUDAPEST